

Universidad Tecnológica de Morelia

9° "B"

Desarrollo Web Integral

4 de agosto de 2026

Alumno:

T.S.U Edgar Antonio Corona Damian

Consulta 1: Mostrar los comentarios creados entre el 20 de junio y el 4 de agosto

Planteamiento de la consulta: Filtrar los comentarios cuyo campo fecha created se encuentre dentro del rango del 20 de junio de 2026 al 4 de agosto de 2026.

```
def consulta_rango_fechas(request):
    fecha_inicio = datetime.date(2026, 6, 20)
    fecha_fin = datetime.date(2026, 8, 4)
    comentarios =
ComentarioContacto.objects.filter(created__range=(fecha_inicio, fecha_fin))
    return render(request, "registros/comentariosregistro.html",
{'comentariocontactos': comentarios})

def consulta_rango_fechas_sql(request):
    tabla = ComentarioContacto._meta.db_table
    comentarios = ComentarioContacto.objects.raw(
        f'''SELECT id, usuario, mensaje, created
        FROM registros_comentariocontacto
        WHERE DATE(created) BETWEEN "2026-06-20" AND "2026-08-04"'''
    )
    return render(request, "registros/comentariosregistro.html",
{'comentariocontactos': comentarios})
```

Principal/consulta_rango_fechas/

Usuario	Mensaje	Editar	Eliminar
lol	:)	Editar	Eliminar
edgar	prueba de modificación	Editar	Eliminar
Juan	pago no aceptado	Editar	Eliminar
edgar555555555555555555	importante	Editar	Eliminar

Principal/consulta_rango_fechas_sql/

Usuario	Mensaje	Editar	Eliminar
edgar555555555555555555	importante	Editar	Eliminar
Juan	pago no aceptado	Editar	Eliminar
edgar	prueba de modificación	Editar	Eliminar
lol	:)	Editar	Eliminar

Consulta 2: Buscar una expresión en el comentario

Planteamiento de la consulta: Filtrar y recuperar únicamente aquellos comentarios que contengan la palabra o expresión especificada dentro del campo mensaje.

```
def consulta_expresion_comentario(request):
    expresion = ";"
    # Se usa mensaje__icontains porque el campo se llama 'mensaje'
    comentarios =
ComentarioContacto.objects.filter(mensaje__icontains=expresion)
    return render(request, "registros/comentariosregistro.html",
{'comentariocontactos': comentarios})

def consulta_expresion_comentario_sql(request):
    tabla = ComentarioContacto._meta.db_table
    comentarios = ComentarioContacto.objects.raw(
        f'''SELECT id, usuario, mensaje, created
        FROM registros_comentariocontacto
        WHERE mensaje LIKE %s''', ['%importante%']
    )
    return render(request, "registros/comentariosregistro.html",
{'comentariocontactos': comentarios})
```

Principal/consulta_expresion_comentario/

Usuario	Mensaje	Editar	Eliminar
lol	;)	<button>Editar</button>	<button>Eliminar</button>

Principal/consulta_expresion_comentario_sql/

Usuario	Mensaje	Editar	Eliminar
edgar55555555555555555555	importante	<button>Editar</button>	<button>Eliminar</button>

Consulta 3: Comentarios pertenecientes a un usuario específico

Planteamiento de la consulta: Obtener todos los comentarios cuyo autor coincida con un usuario en particular registrado en el sistema.

```
def consulta_por_usuario(request):
    usuario_buscado = "lol"
    comentarios = ComentarioContacto.objects.filter(usuario=usuario_buscado)
    return render(request, "registros/comentariosregistro.html",
{'comentariocontactos': comentarios})

def consulta_por_usuario_sql(request):
    tabla = ComentarioContacto._meta.db_table
    comentarios = ComentarioContacto.objects.raw(
        f'''SELECT id, usuario, mensaje, created
        FROM registros_comentariocontacto
        WHERE usuario = %s''', ["edgar"]
    )
    return render(request, "registros/comentariosregistro.html",
{'comentariocontactos': comentarios})
```

Principal/consulta_por_usuario/

Usuario	Mensaje	Editar	Eliminar
lol	:)	<button>Editar</button>	<button>Eliminar</button>

Principal/consulta_por_usuario_sql/

Usuario	Mensaje	Editar	Eliminar
edgar	prueba de modificación	<button>Editar</button>	<button>Eliminar</button>
edgar	pago realizado	<button>Editar</button>	<button>Eliminar</button>

Consulta 4: Filtrar por año de creación

Planteamiento de la consulta: Obtener la lista de comentarios basándose en el año exacto de su fecha de registro.

```
def consulta_expresion_year(request):
    comentarios = ComentarioContacto.objects.filter(created__year=2026)
    return render(request, "registros/comentariosregistro.html",
{'comentariocontactos': comentarios})

def consulta_expresion_year_sql(request):
    tabla = ComentarioContacto._meta.db_table
    comentarios = ComentarioContacto.objects.raw(
        f'''SELECT id, usuario, mensaje, created
        FROM registros_comentariocontacto
        WHERE strftime("%%Y", created) = "2026"'''
    )
    return render(request, "registros/comentariosregistro.html",
{'comentariocontactos': comentarios})
```

[Principal/consulta_expresion_year/](#)

Usuario	Mensaje	Editar	Eliminar
edgar	pago realizado	Editar	Eliminar
lol	;)	Editar	Eliminar
edgar	prueba de modificación	Editar	Eliminar
Juan	pago no aceptado	Editar	Eliminar
edgar555555555555555555	importante	Editar	Eliminar

[Principal/consulta_expresion_year_sql/](#)

Usuario	Mensaje	Editar	Eliminar
edgar555555555555555555	importante	Editar	Eliminar
Juan	pago no aceptado	Editar	Eliminar
edgar	prueba de modificación	Editar	Eliminar
lol	;)	Editar	Eliminar
edgar	pago realizado	Editar	Eliminar

Consulta 5: Búsqueda por prefijo exacto

Planteamiento de la consulta: Buscar los comentarios cuyo texto en el campo mensaje inicie explícitamente con la palabra “pago o prueba”.

```
def consulta_expresion_startswith(request):
    comentarios =
ComentarioContacto.objects.filter(mensaje__startswith='prueba')
    return render(request, "registros/comentariosregistro.html",
{'comentariocontactos': comentarios})

def consulta_expresion_startswith_sql(request):
    tabla = ComentarioContacto._meta.db_table
    comentarios = ComentarioContacto.objects.raw(
        f'''SELECT id, usuario, mensaje, created
        FROM registros_comentariocontacto
        WHERE mensaje LIKE %s''', ['pago%']
    )
    return render(request, "registros/comentariosregistro.html",
{'comentariocontactos': comentarios})
```

[Principal/consulta_expresion_startswith/](#)

Usuario	Mensaje	Editar	Eliminar
edgar	prueba de modificación	<button>Editar</button>	<button>Eliminar</button>

[Principal/consulta_expresion_startswith_sql/](#)

Usuario	Mensaje	Editar	Eliminar
Juan	pago no aceptado	<button>Editar</button>	<button>Eliminar</button>
edgar	pago realizado	<button>Editar</button>	<button>Eliminar</button>

Conclusión

Ventajas

ORM

- Abstracción del motor de base de datos: El código en Python es independiente del gestor (SQLite, PostgreSQL, MySQL).
- Seguridad integrada: Protege la aplicación sanitizando parámetros para evitar inyecciones SQL.
- Velocidad de desarrollo: Permite interactuar con la base de datos usando objetos y sintaxis nativa de Python.

SQL

- Control total: Permite ejecutar consultas complejas o muy optimizadas que no son sencillas de construir en el ORM.
- Sintaxis nativa: Permite aprovechar funciones específicas del gestor de base de datos.

Desventajas

ORM

- Costo de procesamiento: Existe una ligera sobrecarga al traducir las llamadas ORM a sentencias SQL.
- Curva de aprendizaje de la API: Consultas avanzadas requieren dominar operadores específicos de la API de Django.

SQL

- Riesgos de seguridad: Si los parámetros no se pasan adecuadamente, la aplicación queda expuesta a ataques de inyección SQL.
- Requisito de llave primaria: Exige incluir obligatoriamente el campo id en la selección para instanciar los modelos.

¿Por qué Django apuesta por el ORM?

Django apuesta por el ORM para seguir el principio DRY y mantener la separación de responsabilidades dentro del patrón MVT. Centraliza la estructura de los datos en clases de Python, previene fallas de seguridad por inyección de código y evita acoplar el proyecto a un motor de base de datos particular, facilitando el mantenimiento y la escalabilidad de la aplicación.

